

F A T E C

P1 — Relatório Bimestral de Atividades

Cine Mágico: do monolito à arquitetura de microsserviços

Aluno: Cleiton Guilherme Rodrigues de Souza

Disciplina: Introdução à Computação em Nuvem

Turma: Sistemas Inteligentes

Semestre: 2026/2

Entrega: 07/10/2026

1. Introdução

A disciplina de Introdução à Computação em Nuvem teve como objetivo prático a construção progressiva de uma aplicação baseada em microsserviços. Ao longo do bimestre, desenvolvemos o "Cine Mágico", um catálogo de filmes integrado à API do TMDb que evoluiu de um simples monolito para uma arquitetura distribuída com serviços de autenticação, banco de dados (MySQL), filas de eventos (Redis), controle de acesso (RBAC) e armazenamento de objetos (MinIO). Este relatório consolida e comprova todas as entregas realizadas durante esse período, servindo como uma prestação de contas técnica do desenvolvimento.

2. Metodologia

Todas as atividades foram desenvolvidas localmente utilizando o Docker e Docker Compose para garantir a padronização e orquestração dos contêineres (Node.js, MySQL, Redis, MinIO). O versionamento do código foi feito via Git, com repositório remoto hospedado no GitHub. A interface foi construída em React (Vite) e as APIs em Node.js (Express).

As datas e horas das evidências apresentadas neste relatório foram extraídas diretamente do histórico de commits do GitHub (`git log`), refletindo o momento em que o código funcional foi consolidado no repositório. Os prints foram capturados no ambiente de desenvolvimento (localhost) com o sistema rodando integralmente no Docker.

3. Quadro de Entregas

Nº	Atividade / Descrição	Data Planejada	Data Realizada	Situação
1	Agenda telefônica em Flask	07/08/2026	07/08/2026	Entregue
2	Catálogo de filmes — Tom Hanks	20/08/2026	20/08/2026 21:06	Entregue
3	Desacoplando o login — microsserviço	28/08/2026	26/08/2026 21:07	Entregue
4	Controle de acesso por papel — RBAC	04/09/2026	03/09/2026 19:52	Entregue
5	Logs e auditoria	25/09/2026	06/10/2026 10:52	Entregue com atraso
6	Upload e perfil de usuário	02/10/2026	06/10/2026 11:54	Entregue com atraso

```
git log (repositório cleitongrodrigues/Filmes)

ff5b4d8 06/10/2026 11:56 docs: atualiza documentacao com arquitetura da atividade 6
46cc1b8 06/10/2026 11:55 feat: implementa interface de perfil do usuario e avatar no nav
c011938 06/10/2026 11:55 feat: cria endpoint de perfil, regras de upload e migration do db
96def38 06/10/2026 11:54 feat: integra MinIO s3 para armazenamento de arquivos e fotos
7fa718f 06/10/2026 10:53 docs: atualiza README.md com diagramas e requisitos da Atividade 5 (Logs e Auditoria)
00d97bd 06/10/2026 10:53 feat(auditoria): injeta disparo de eventos nas ações de negocio e controle de acesso
6506663 06/10/2026 10:52 feat(admin): implementa rota de consulta do histórico de logs via backend
bbd7065 06/10/2026 10:52 feat(logs): adiciona axios e cliente de integração com o log-service
d429cdb 06/10/2026 10:52 feat(infra): adiciona microserviço log-service e Redis no docker-compose
038f1a8 03/09/2026 19:59 feat: Alterando o README
d32ce6d 03/09/2026 19:59 feat(frontend): exhibe papel do usuário e feedback da moderação
be9576b 03/09/2026 19:52 feat(backend): aplica RBAC na moderação de comentários
063821c 26/08/2026 21:07 docs: atualizar documentacao e configuracoes
6feeea6 26/08/2026 21:07 feat: adicionar microserviço de autenticação
0d97fa8 20/08/2026 21:06 Adicionadno o tom hanks
9d2f4c0 20/08/2026 21:00 configurando
8f0f0b8 20/08/2026 20:44 fix: ajuste na API
7c29f21 20/08/2026 20:38 Adicionado a API de filmes
886301c 20/08/2026 20:22 Subindo ajuste no docker
3d16201 20/08/2026 20:19 Apontando para a base de dados
5186e63 20/08/2026 20:11 Configurando para o deploy
7f709bb 20/08/2026 19:56 Subindo o projeto para o git
86570cf 20/08/2026 19:36 Initial commit
```

Figura 3.0 — Histórico de commits que sustenta as datas do quadro.

4. Atividades Realizadas

Ficha — Atividade 1: Agenda telefônica em Flask

Data planejada	07/08/2026	Data realizada	07/08/2026
Situação	Entregue		
Evidência	Repositório do projeto: https://github.com/cleitongrodrigues/Filmes		

Descrição: Agenda telefônica em Flask desenvolvida em sala de aula.

O que foi feito: Configuração inicial de um servidor Flask simples para simular uma agenda telefônica, familiarizando com conceitos básicos de rotas e web.

Dificuldades: Instalação inicial das dependências do Python no ambiente, resolvida com a criação de um ambiente virtual (venv).

Atividade realizada em sala, em repositório separado do projeto de filmes; o commit de entrega não consta no histórico deste repositório.

Ficha — Atividade 2: Catálogo de filmes — Tom Hanks

Data planejada	20/08/2026	Data realizada	20/08/2026 21:06
Situação	Entregue		
Evidência	github.com/cleitongrodrigues/Filmes/commit/0d97fa8		

Descrição: Criação do catálogo de filmes do Tom Hanks, consumindo a API do TMDB.

O que foi feito: Desenvolvimento do front-end em React e back-end em Node.js. O sistema busca dados do TMDB e salva favoritos no MySQL de produção na nuvem.

Dificuldades: Gerenciar as variáveis de ambiente e a chave do TMDB com segurança, resolvido utilizando o pacote dotenv no backend e o import.meta.env no Vite.

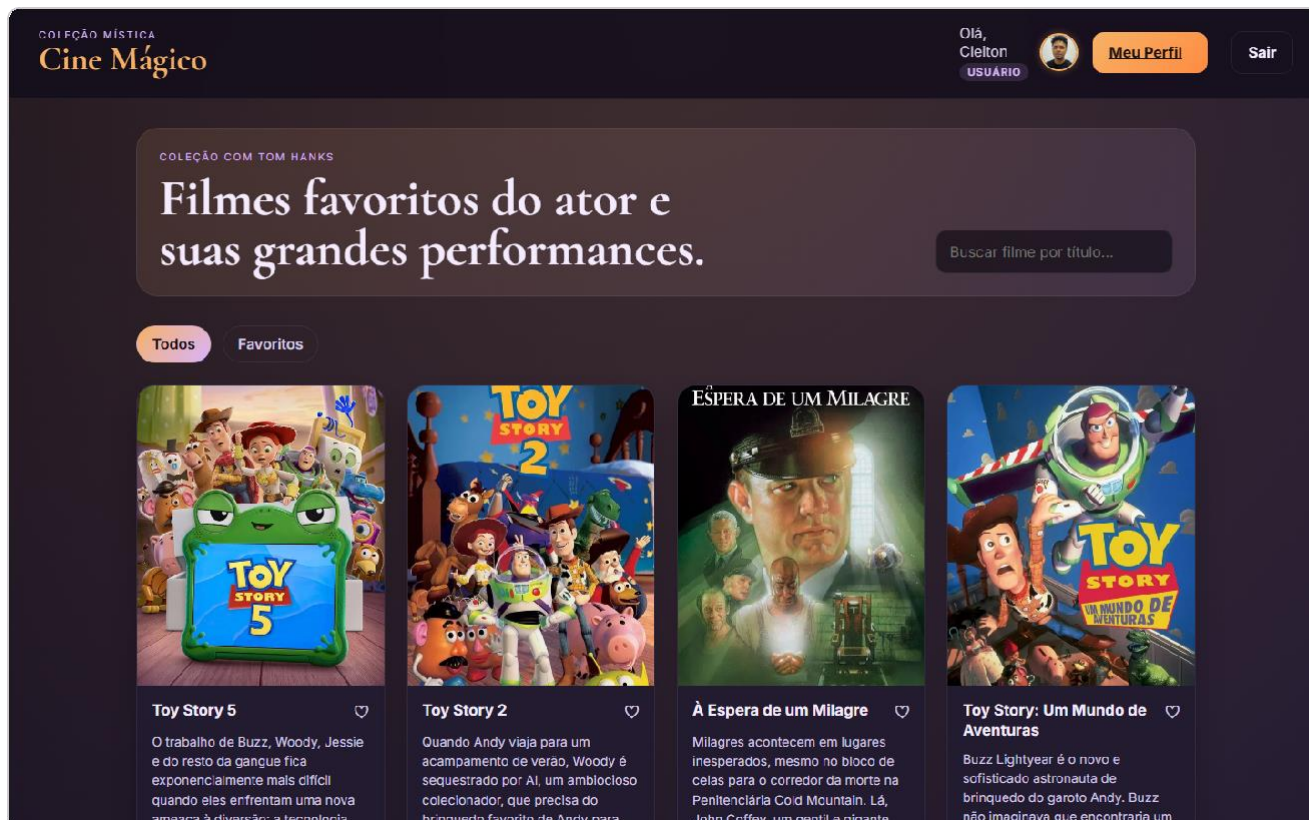


Figura 2.1 — Catálogo de filmes com a coleção do Tom Hanks (sistema rodando em Docker, localhost:8201).

Ficha — Atividade 3: Desacoplando o login — microserviço

Data planejada	28/08/2026	Data realizada	26/08/2026 21:07
Situação	Entregue		
Evidência	github.com/cleitongrodrigues/Filmes/commit/6feeea6		

Descrição: Desacoplando o login para um microserviço de autenticação próprio.

O que foi feito: Separação completa da lógica de JWT e Bcrypt para um contêiner Docker isolado (auth-service), que se comunica com o backend principal apenas via requisições HTTP internas.

Dificuldades: Fazer os contêineres conversarem na rede interna do Docker. Resolvido configurando uma bridge network dedicada (cine-network) no docker-compose.yml.

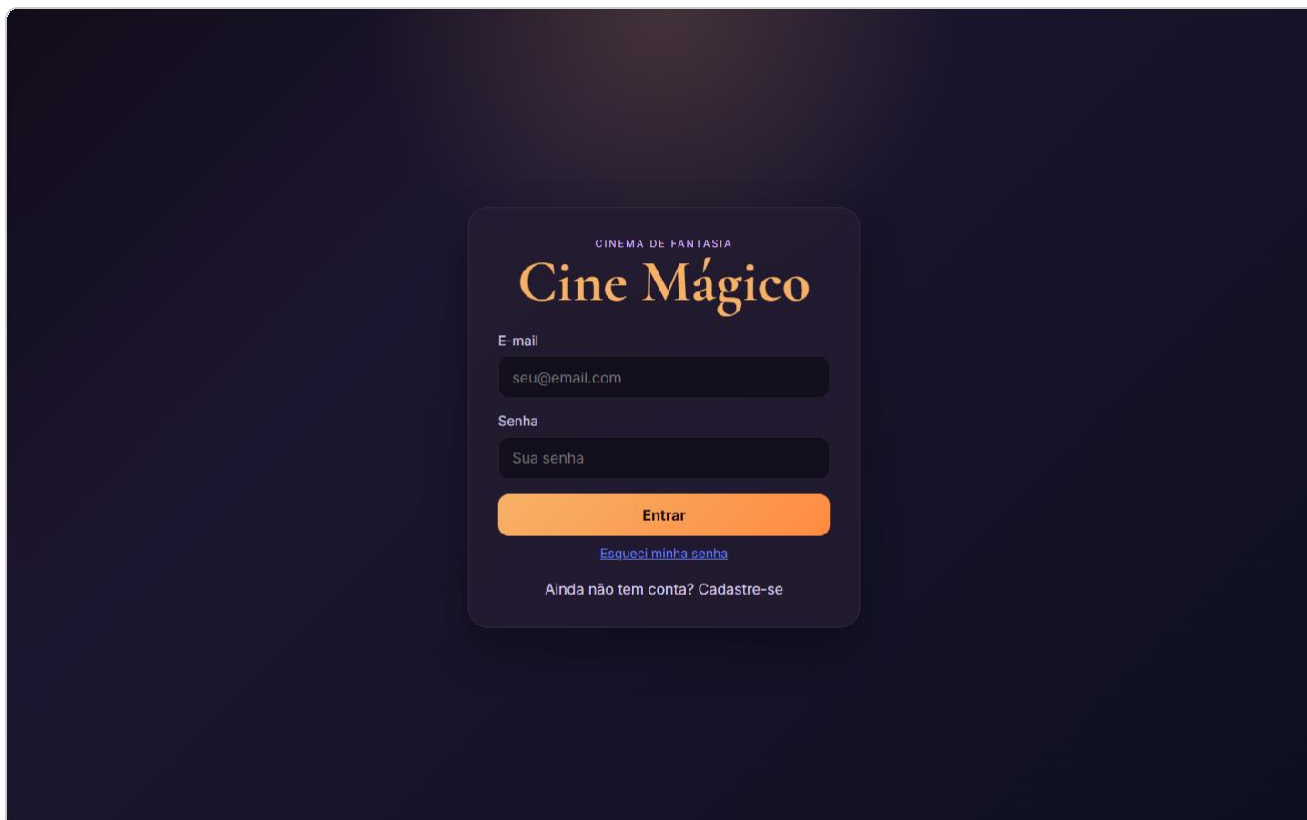


Figura 3.1 — Tela de login atendida pelo auth-service.

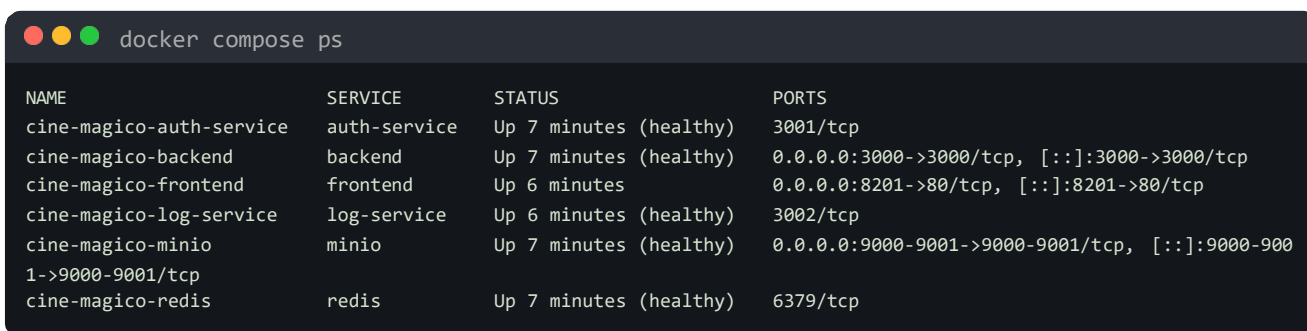


Figura 3.2 — Serviços ativos, incluindo o contêiner auth-service isolado.

Ficha — Atividade 4: Controle de acesso por papel — RBAC

Data planejada	04/09/2026	Data realizada	03/09/2026 19:52
Situação	Entregue		
Evidência	github.com/cleitongrodrigues/Filmes/commit/be9576b github.com/cleitongrodrigues/Filmes/commit/d32ce6d		

Descrição: Controle de acesso por papel (RBAC), limitando ações ao administrador.

O que foi feito: Adicionada a coluna role no banco de dados e criado um middleware de autorização que bloqueia rotas sensíveis (como a auditoria e a deleção de comentários de terceiros) caso o token não possua a role admin, retornando erro HTTP 403.

Dificuldades: Propagar corretamente a autorização (role) para o front-end esconder botões de edição. Resolvido injetando a claim da role diretamente dentro do payload do token JWT no ato do login.

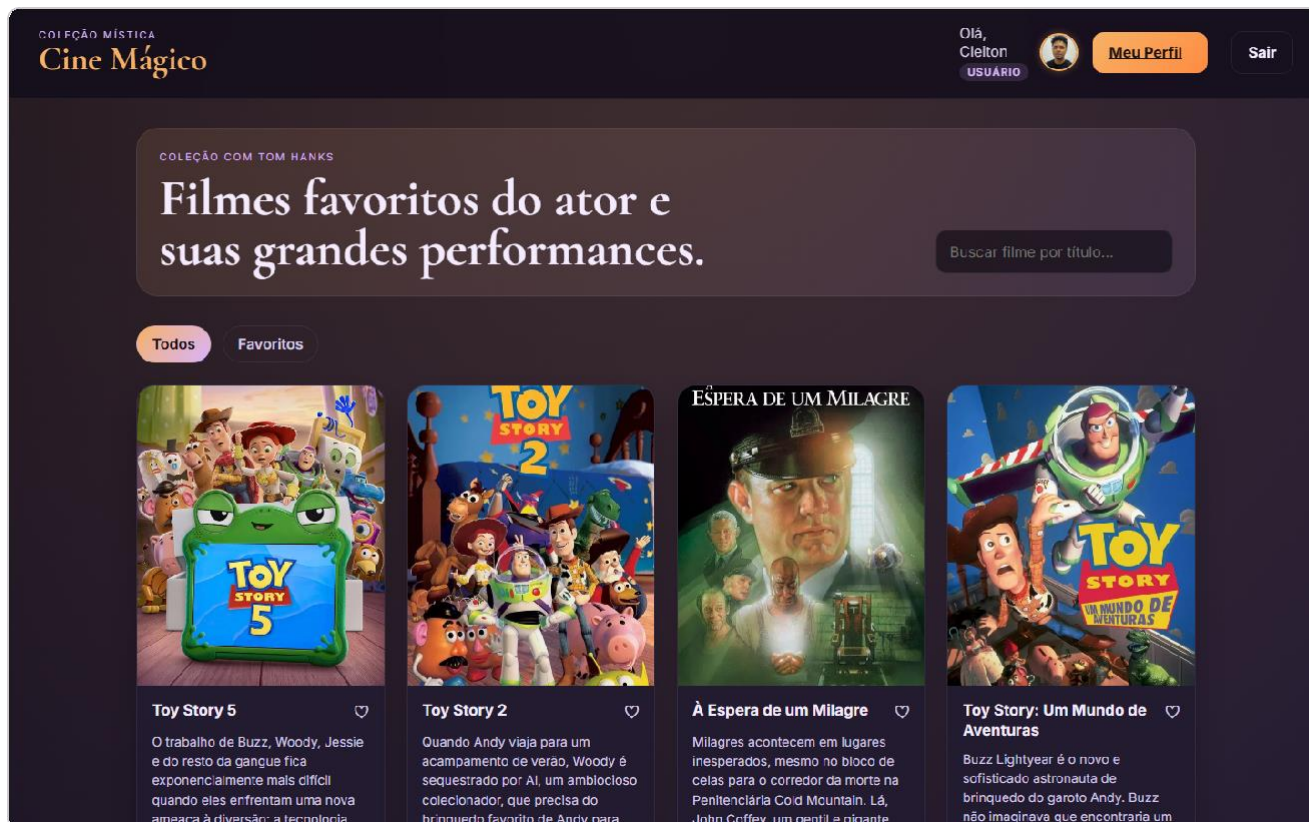


Figura 4.1 — Cabeçalho do catálogo exibindo o papel do usuário (“Usuário”), propagado pela claim role do JWT.

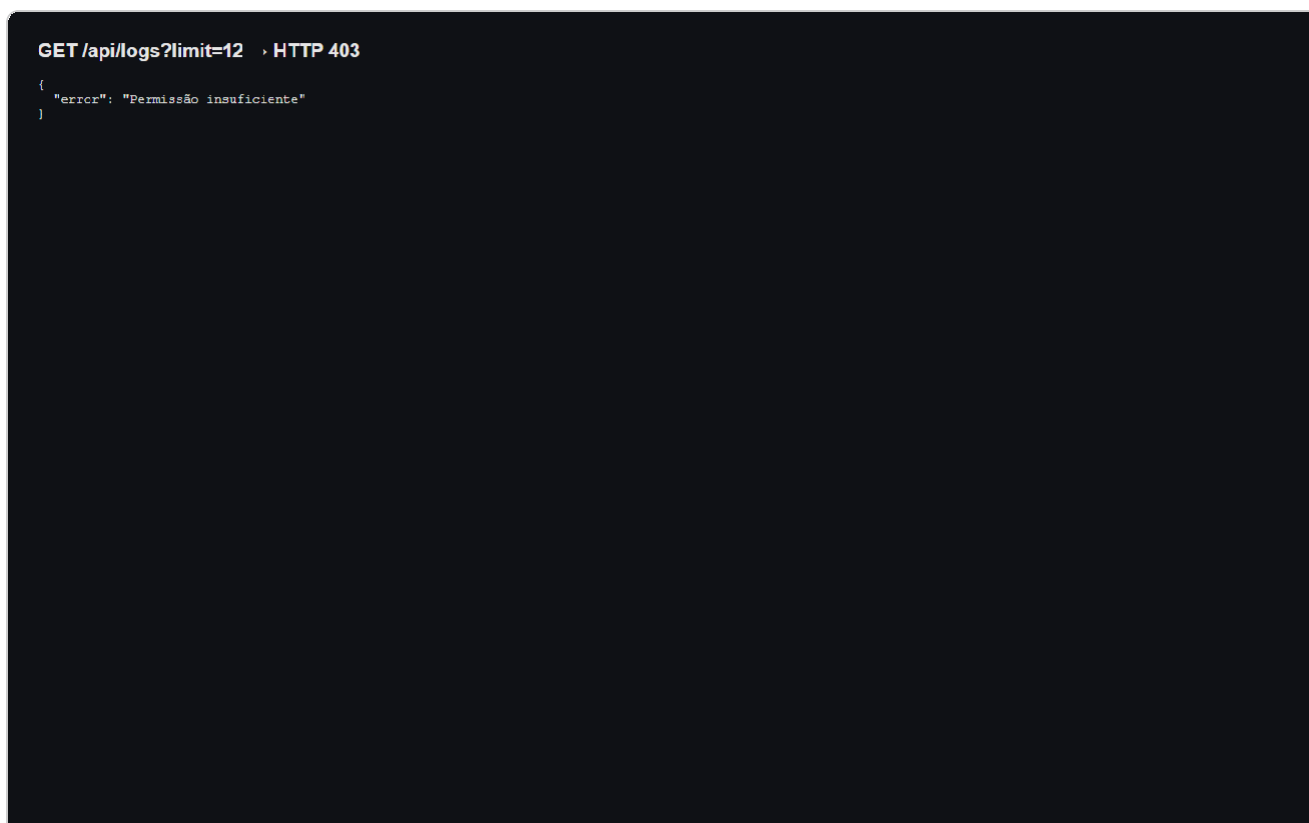


Figura 4.2 — Requisição GET /api/logs com usuário de papel “usuário”: o backend responde HTTP 403 “Permissão insuficiente”.

Ficha — Atividade 5: Logs e auditoria

Data planejada	25/09/2026	Data realizada	06/10/2026 10:52
----------------	------------	----------------	------------------

Situação	Entregue com atraso
Motivo do atraso	[Inserir o motivo real do atraso desta atividade]
Evidência	github.com/cleitongrodrigues/Filmes/commit/d429cdb github.com/cleitongrodrigues/Filmes/commit/7fa718f

Descrição: Criação do serviço de logs e auditoria isolado com Redis.

O que foi feito: Implementação de um terceiro microserviço isolado (log-service) que consome streams do Redis (XADD/XREADGROUP) enviados pelo backend principal para auditar permanentemente ações de segurança, falhas de login e erros 403.

Dificuldades: Entender o padrão de Streams do Redis comparado ao Pub/Sub convencional e evitar que os logs fossem perdidos se o serviço caísse. Resolvido consultando a documentação oficial para garantir Consumer Groups persistentes.

```

docker logs cine-magico-log-service

> log-service@1.0.0 start
> node server.js

ÔÊÔ Conectado ao Redis
ÔÊÔ Log Service rodando na porta 3002

```

Figura 5.1 — log-service em execução, conectado ao Redis.

```

GET /api/logs?limit=12 → HTTP 403

{
  "error": "Permissão insuficiente"
}

```

Figura 5.2 — Requisição GET /api/logs com usuário de papel "usuário": o backend responde HTTP 403 "Permissão insuficiente".

Ficha — Atividade 6: Upload e perfil de usuário

Data planejada	02/10/2026	Data realizada	06/10/2026 11:54
-----------------------	------------	-----------------------	------------------

Situação	Entregue com atraso
Motivo do atraso	Problemas críticos de rede com imagens do Docker Hub (o MinIO exigiu uma build espelhada da Elestio) e falhas de assinatura criptográfica "SignatureMismatch" na geração dos links temporários de exibição da imagem, o que exigiu muito tempo de depuração na infraestrutura.
Evidência	github.com/cleitongrodrigues/Filmes/commit/96def38 github.com/cleitongrodrigues/Filmes/commit/ff5b4d8

Descrição: Upload e armazenamento da foto de perfil com segurança em Object Storage.

O que foi feito: Integração do backend com um contêiner MinIO (S3-compatible) local utilizando o pacote @aws-sdk. Lógica de Pre-signed URLs com expiração de 1 hora para proteger o bucket do acesso público, além da tela de perfil (Profile.jsx) limitando tamanho (5MB) e tipo de arquivo (somente imagem).

Dificuldades: O navegador do cliente não conseguia baixar a imagem pela URL assinada internamente pelo Node.js (http://minio:9000). Para resolver o hostname do Docker sem invalidar a assinatura, foram usados dois clients S3: um apontando para minio:9000 (upload interno) e um s3PresignClient apontando para localhost:9000 (geração dos links).

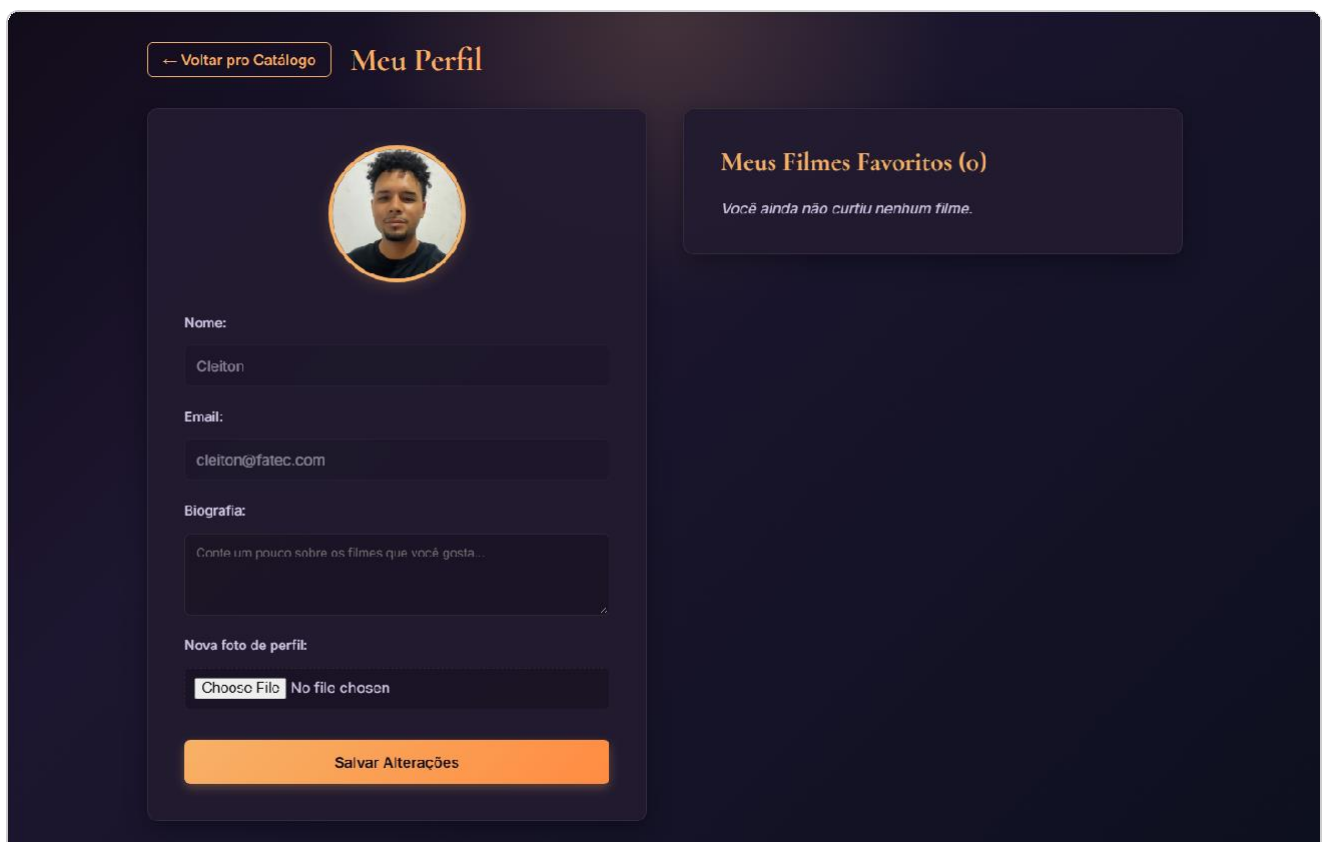


Figura 6.1 — Página de perfil com a foto carregada do MinIO via URL pré-assinada.

Figura 6.2 — O avatar também aparece no cabeçalho do catálogo (Figura 2.1), ao lado do papel "Usuário".

5. Considerações Finais

A jornada de arquitetura em nuvem ao longo deste bimestre foi extremamente técnica, escalável e desafiadora. O maior aprendizado prático não foi apenas escrever código funcional, mas entender

sistemicamente como integrar e blindar sistemas distribuídos: desde orquestrar o tráfego em bridge networks no Docker, gerenciar transações e filas assíncronas no Redis, até o forte isolamento de buckets S3 usando URLs criptografadas que expiram. A barreira de maior complexidade foi mapear corretamente a comunicação local entre os microsserviços e o host, mas foi o que consolidou minha visão realista da engenharia de Cloud Computing e DevOps.

Declaração de Autoria

Declaro que este relatório é de minha autoria, individual e original, e que as evidências fornecidas nos links do repositório GitHub correspondem fielmente aos commits e linhas de código de fato enviadas e validadas por mim nas datas e horas informadas.

Cleiton Guilherme R. De Souza